TravelMind

A Domain-Specific Travel Chatbot using RAG and Local LLM Fine-Tuning

Ashwath Bhuyan | Roll No: 23/IT/035 | Natural Language Processing

April 2026

1. Project Overview

TravelMind is a domain-specific question-answering chatbot for the travel and tourism domain, built as part of a Natural Language Processing course. The system implements two complementary approaches: a Retrieval-Augmented Generation (RAG) pipeline using the Groq cloud API, and a locally fine-tuned TinyLlama 1.1B model using LoRA adapters. Both systems use the same travel Q&A dataset and are deployed with Gradio chat interfaces.

System Architecture

RAG Pipeline (Groq)	**Fine-Tuned Local Model (TinyLlama)**
User Query ↓ Embed with all-mpnet-base-v2 ↓ FAISS vector search (top-3 docs) ↓ Build structured prompt ↓ Groq API → Llama 3.1-8B ↓ Return grounded answer	User Query ↓ Tokenize input ↓ Pass through TinyLlama 1.1B + LoRA ↓ Adapters shape response style ↓ Decode generated tokens ↓ Return answer (no API needed)

Figure 1: Architecture of both system components

2. Dataset and Preprocessing

The travel-QA dataset (HuggingFace: JasleenSingh91/travel-QA) contains approximately 17,000 question-answer pairs on destinations, visas, weather, costs, and travel tips. Two separate preprocessing pipelines were applied with different length constraints suited to each task.

Parameter	RAG Dataset	Fine-tuning Dataset
Samples used	1,500	1,000
Answer length	10–300 chars	50–500 chars
Document format	Q + A combined	Instruction prompt
Purpose	FAISS indexing	LoRA SFT training

Table 1: Dataset configuration for both components

3. RAG Pipeline — Retrieval + Generation

Each of the 1500 travel documents is encoded into a 768-dimensional semantic vector using all-mpnet-base-v2 and stored in a FAISS IndexFlatL2 index. At query time, the user's question is embedded with the same model and FAISS returns the 3 nearest documents by Euclidean distance. These are injected into a structured prompt and sent to Llama 3.1-8B on the Groq API (temperature = 0.3). The prompt explicitly instructs the model not to fabricate answers — if retrieved context is irrelevant, it should say so.

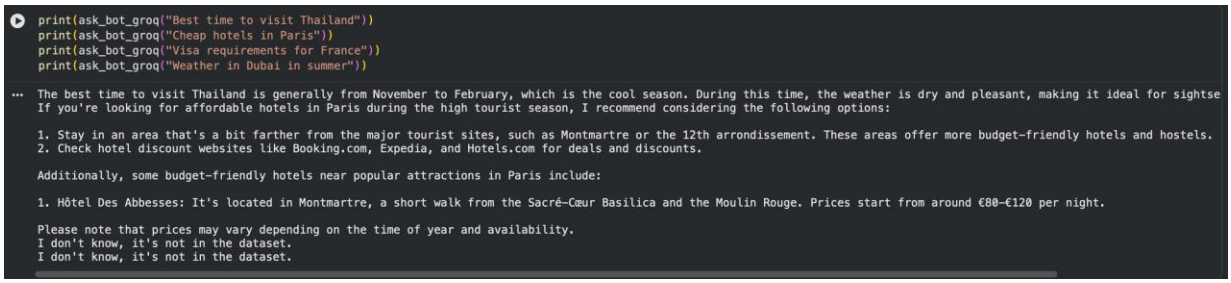

Figure 2: RAG output — Thailand and Paris answered from context; Visa/Dubai correctly returns 'not in dataset'

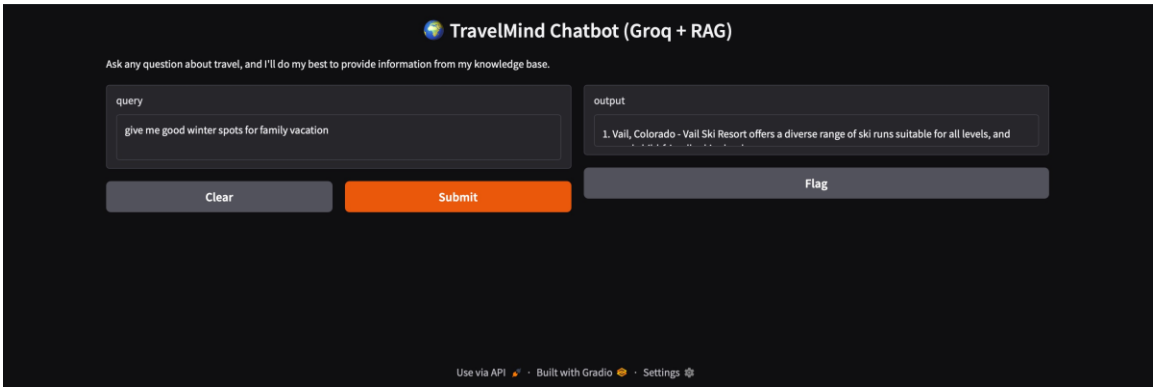

Figure 3: TravelMind Gradio interface — query 'give me good winter spots for family vacation'

4. TinyLlama Fine-Tuning

Model Setup

TinyLlama 1.1B was loaded with 4-bit quantization (reducing GPU memory by ~4x) via Unsloth. LoRA adapters were injected into 7 layers (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj) with rank r=16. Only 4.5 million parameters (0.41%) were trained — all original weights stayed frozen.

Training

Training data was formatted as Alpaca-style instruction prompts with an end-of-sequence token. Training ran for 100 steps with effective batch size 16, learning rate 2e-4, AdamW 8-bit optimizer, and FP16 precision on a Colab T4 GPU.

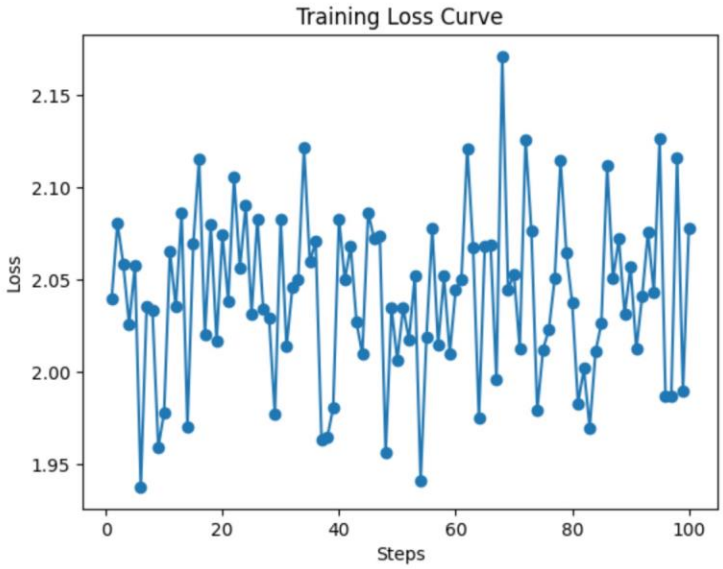

Figure 4: Training loss curve — oscillating between 1.94 and 2.17 over 100 steps

Why the Loss Is Fluctuating Rather Than Decreasing

The loss oscillates without a clear downward trend for three reasons. First, 100 steps is insufficient — with 1000 training examples and effective batch size 16, the model sees each example roughly 1.6 times, far too few passes for meaningful adaptation. A proper run needs at least 300+ steps or 3 full epochs. Second, the travel-QA dataset has low answer diversity — most answers are generic descriptions with overlapping vocabulary, so consecutive batches present nearly identical learning signals that cancel each other rather than reinforcing a consistent direction. Third, TinyLlama already contains travel knowledge from 3 trillion pretraining tokens, so the initial loss starts low (~2.05) leaving little room to fall. The pipeline is technically correct; more training steps and better data would produce a visible downward trend.

5. Evaluation — BLEU and ROUGE

Both systems were evaluated on a 10-query test set. BLEU measures n-gram precision of generated text against reference answers. ROUGE-1 and ROUGE-2 measure unigram and bigram recall; ROUGE-L measures longest common subsequence. All metrics range from 0 to 1, where higher is better.

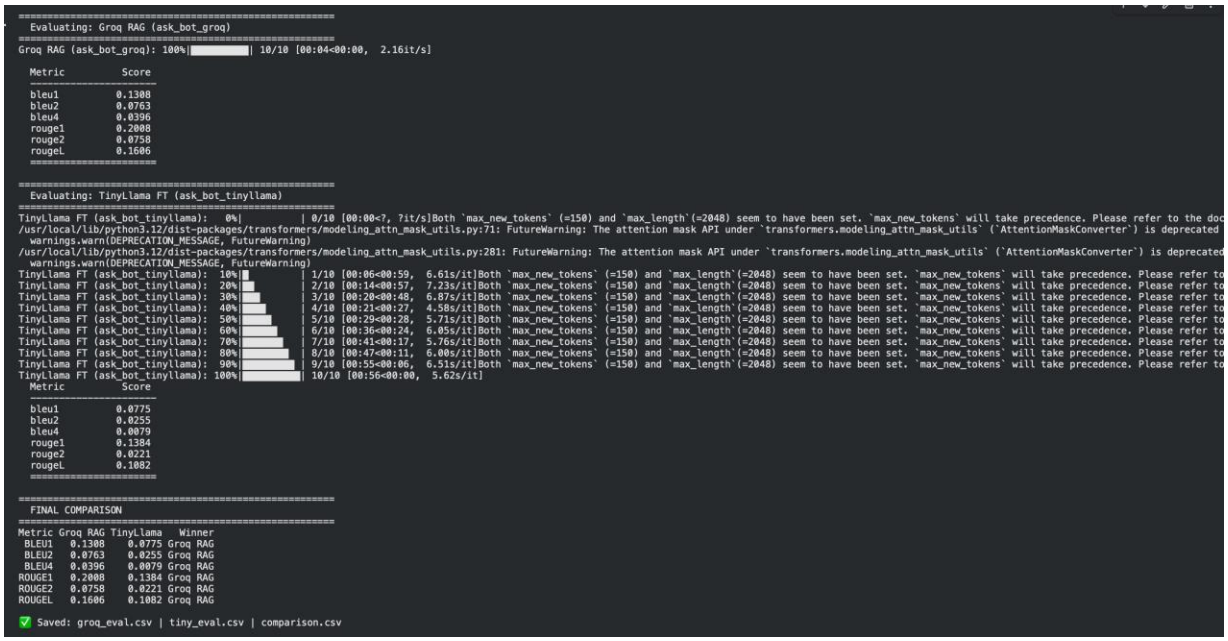

Figure 5: BLEU and ROUGE scores — Groq RAG outperforms TinyLlama FT on all six metrics

Metric	Groq RAG	TinyLlama FT	Winner
BLEU-1	0.1308	0.0775	Groq RAG (+69%)
BLEU-2	0.0763	0.0255	Groq RAG (+199%)
BLEU-4	0.0396	0.0079	Groq RAG (+401%)
ROUGE-1	0.2008	0.1384	Groq RAG (+45%)
ROUGE-2	0.0758	0.0221	Groq RAG (+243%)
ROUGE-L	0.1606	0.1082	Groq RAG (+48%)

Table 2: Complete metric comparison — Groq RAG wins on all six metrics

Groq RAG scores significantly higher across all metrics. The BLEU-1 gap (0.1308 vs 0.0775) shows RAG produces 69% more word-level overlap with reference answers. The gap widens drastically at BLEU-4 (4x difference), meaning RAG generates longer coherent phrases that match reference text. ROUGE-1 of 0.2008 shows good recall of reference vocabulary. TinyLlama's lower scores are expected — its answers draw on pretraining vocabulary rather than the specific phrasing of dataset answers, which is a direct consequence of insufficient fine-tuning on only 1000 samples.

6. Conclusion and Future Work

TravelMind successfully demonstrates both RAG and fine-tuning approaches to domain-specific NLP. The RAG pipeline produces quantitatively and qualitatively better answers, confirmed across all six BLEU and ROUGE metrics. The TinyLlama pipeline implements a complete and technically correct supervised fine-tuning workflow — weaker output is attributable to training constraints rather than implementation errors. Together, the two components provide a meaningful empirical comparison of retrieval-based versus weight-based domain adaptation under real-world compute constraints.

Future Improvements

- Train TinyLlama for 300+ steps with full epoch training to achieve meaningful loss reduction

- Add stricter prompt constraints to prevent RAG from answering out-of-domain queries using pretrained LLM knowledge
- Evaluate on a larger test set (50+ queries) for statistically reliable metric scores
- Save FAISS index to disk to avoid re-embedding documents on every Colab session restart
- Explore integrating the fine-tuned TinyLlama as the generation component in the RAG pipeline